

ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

Bài giảng cho sinh viên ngành Công nghệ thông tin

Đỗ Thị Bích Ngọc

Hà Nội - 2020

MỞ ĐẦU

Trước những thách thức trong quá trình phát triển phần mềm, việc đảm bảo chất lượng phần mềm (Software Quality Assurance-SQA) là hết sức quan trọng, đòi hỏi phải nghiên cứu một cách nghiêm túc để thực thi hiệu quả. Tài liệu này cung cấp những kiến thức cơ bản về chất lượng phần mềm, đảm bảo chất lượng trong một dự án phát triển phần mềm. Quy trình xây dựng hệ thống đảm bảo chất lượng phần mềm cũng được trình bày trong nội dung bài giảng. Qua đó, sinh viên hiểu được cách thức xây dựng một hệ thống đảm bảo chất lượng phần mềm và vai trò của những thành viên trong hệ thống. Một số chuẩn đảm bảo chất lượng cũng được giới thiệu trong chương cuối. Thông qua nội dung bài giảng sinh viên cũng sẽ nắm được kỹ năng rà soát và kiểm thử phần mềm.

Nội dung bài giảng được xây dựng trong bảy chương:

Chương 1. Giới thiệu đảm bảo chất lượng phần mềm

Những khái niệm mở đầu của tài liệu được giới thiệu trong Chương 1. Bắt đầu với khái niệm phần mềm, chất lượng phần mềm và đảm bảo chất lượng phần mềm, phần tiếp theo phân tích các tiêu chí chất lượng phần mềm.

Chương 2. Tích hợp các hoạt động đảm bảo chất lượng phần mềm vào vòng đời phát triển phần mềm

Chương 2 đề cập đến các thành phần đảm bảo chất lượng phần mềm trong vòng đời dự án phần mềm. Những nội dung được trình bày trong chương này bao gồm : phân tích một số mô hình phát triển phần mềm phổ biến. Sau đó, chương 2 đề cập đến các mức độ kiểm thử phần mềm.

Chương 3. Các hoạt động rà soát

Chương 3 trình bày về hoạt động rà soát cho các loại tài liệu được tạo trong quá trình phát triển phần mềm. Chương 3 trình bày các nguyên tắc và phương pháp thực hiện rà soát cũng như một số checklist rà soát mẫu.

Chương 4. Kiểm thử phần mềm

Chương 4 đề cập đến các khái niệm cơ bản trong kiểm thử phần mềm. Những nội dung được trình bày trong chương này bao gồm : khái niệm cơ bản, các mức kiểm thử, quá trình kiểm thử, cũng như ca kiểm thử.

Chương 5: Kỹ thuật kiểm thử hộp đen và hộp trắng

Chương này trình bày 2 kỹ thuật chính dùng trong thiết kế ca kiểm thử. Các kỹ thuật kiểm thử hộp đen để kiểm thử chức năng, nghiệp vụ của hệ thống. Các kỹ thuật kiểm thử hộp trắng để kiểm thử code, kiểm thử đơn vị.

Chương 6. Các công cụ hỗ trợ đảm bảo chất lượng phần mềm

Chương 6 đề cập đến các loại công cụ được dùng trong kiểm thử phần mềm. Những nội dung được trình bày trong chương này bao gồm : các phần mềm phục vụ quản lý kiểm thử, các công cụ hỗ trợ kiểm thử, và các công cụ hỗ trợ kiểm thử tự động cho cả kiểm thử chức năng và kiểm thử phi chức năng.

Chương 7. Các chuẩn, chứng chỉ và hoạt động đánh giá

Chương này đề cập tới các chuẩn quản lý chất lượng như ISO, CMM/CMML, trong đó đi sâu vào CMM/CMML.

Phụ lục

Gồm 3 phụ lục :

- Trình bày về các lỗi thường gặp khi viết chương trình.
- Trình bày một số hướng dẫn cho các loại kiểm thử
- Một test plan mẫu

CHƯƠNG 1: GIỚI THIỆU ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM	7
1.1 Khái niệm phần mềm	7
1.2 Các nguyên nhân gây ra lỗi phần mềm	8
1.2.1 Một số ví dụ điển hình về lỗi phần mềm	8
1.2.2 Lỗi phần mềm	13
1.2.3 Nguyên nhân gây ra lỗi phần mềm	14
1.3 Đảm bảo chất lượng phần mềm.....	17
1.3.1 Khái niệm.....	17
1.3.2 Mục tiêu đảm bảo chất lượng phần mềm	18
1.3.3 Xác minh, thẩm định và đánh giá chất lượng	18
1.4 Các tiêu chí chất lượng	19
1.5 Các tiêu chí chất lượng ảnh hưởng tới hoạt động đảm bảo chất lượng phần mềm như nào.	23
 CHƯƠNG 2: TÍCH HỢP CÁC HOẠT ĐỘNG ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM VÀO VÒNG ĐỜI PHÁT TRIỂN PHẦN MỀM.....	 25
2.1 Các phương pháp phát triển phần mềm.....	25
2.2 Các hoạt động đảm bảo chất lượng phần mềm.....	29
2.2.1 Đảm bảo chất lượng hợp đồng.....	29
2.2.2 Đảm bảo chất lượng đặc tả	30
2.2.3 Đảm bảo chất lượng phân tích, thiết kế.....	32
2.2.4 Đảm bảo chất lượng phát triển phần mềm (lập trình)	33
2.3 Các mức độ kiểm thử.....	34
2.3.1 Giới thiệu	34
2.3.2 Kiểm thử đơn vị	34
2.3.3 Kiểm thử tích hợp	35
2.3.4 Kiểm thử hệ thống.....	40
2.3.5 Kiểm thử chấp nhận	43
 CHƯƠNG 3: CÁC HOẠT ĐỘNG RÀ SOÁT.....	 44
3.1 Mục tiêu của rà soát	44
3.1.1 Định nghĩa.....	44
3.1.2 Mục tiêu.....	44
3.2 Các hình thức rà soát	44
3.2.1 Rà soát chính thức.....	44
3.2.2 Rà soát ngang hàng.....	47
3.2.3 Ý kiến chuyên gia	48
3.2.4 So sánh rà soát chính thức và rà soát ngang hàng	49
3.3 Thực hiện hoạt động rà soát trong dự án	50
3.3.1 Rà soát hợp đồng.....	50
3.3.2 Rà soát phân tích thiết kế	53
3.3.3 Các hoạt động rà soát khác.....	56
3.4 Đảm bảo chất lượng của các thành phần bảo trì phần mềm.....	63
3.4.1 Giới thiệu	63

3.4.2	Cơ sở cho chất lượng bảo trì cao.....	64
3.4.3	Các thành phần chất lượng phần mềm tiền bảo trì.....	66
3.4.4	Hỗ trợ đảm bảo chất lượng bảo trì phần mềm	70
3.5	Đảm bảo chất lượng phần mềm của các yếu tố bên ngoài cùng tham gia	78
3.5.1	Những thành phần bên ngoài đóng góp vào dự án phần mềm.....	78
3.5.2	Rủi ro và lợi ích của giới thiệu người tham dự ngoài.....	79
3.5.3	Những mục tiêu đảm bảo chất lượng về sự đóng góp người tham gia bên ngoài.....	80
3.5.4	Các công cụ đảm bảo chất lượng những đóng góp của các thành viên đóng góp bên ngoài.....	81
CHƯƠNG 4:	KIỂM THỬ PHẦN MỀM	83
4.1	Định nghĩa và mục tiêu	83
4.1.1	Định nghĩa.....	83
4.1.2	Các mức độ kiểm thử	84
4.2	Quy trình kiểm thử	85
4.2.1	Quy trình.....	85
4.2.2	Input/Output cho test	87
4.2.3	Quản lý lỗi.....	88
4.3	Kế hoạch kiểm thử.....	90
4.4	Thiết kế test (test design).....	92
CHƯƠNG 5:	KỸ THUẬT KIỂM THỬ HỘP ĐEN VÀ HỘP TRẮNG	95
5.1	Các kỹ thuật kiểm thử hộp đen	95
5.1.1	Phân lớp tương đương	95
5.1.2	Kiểm thử biên	99
5.1.3	Bảng quyết định.....	100
5.1.4	Lược đồ chuyển trạng thái.....	101
5.1.5	Kiểm thử theo cặp	103
5.2	Các kỹ thuật kiểm thử hộp trắng.....	106
5.2.1	Kiểm thử luồng điều khiển	106
5.2.2	Kiểm thử luồng dữ liệu	113
5.3	Kiểm thử đơn vị tự động	117
5.3.1	Giới thiệu chung	117
5.3.2	Tổng quan thư viện Junit	119
5.4	Bảng tóm tắt Testing Levels/ Techniques	122
CHƯƠNG 6:	CÁC CÔNG CỤ HỖ TRỢ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM	123
6.1	Các công cụ quản lý thông tin trong Đảm bảo chất lượng phần mềm	123
6.1.1	Phần mềm hỗ trợ viết tài liệu	123
6.1.2	Phần mềm quản lý lỗi	123
6.2	Công cụ kiểm thử tự động là gì ?.....	125
6.2.1	Khái niệm	125
6.2.2	Quy trình kiểm thử tự động.....	125
6.3	Công cụ hỗ trợ kiểm thử đơn vị	126

6.4 Công cụ hỗ trợ kiểm thử chức năng tự động.....	126
6.4.1 Selenium WebDriver.....	129
6.4.2 Các câu lệnh sử dụng trong Selenium WebDriver	130
6.5 Công cụ hỗ trợ kiểm thử API.....	132
Giới thiệu công cụ kiểm thử API Postman	134
6.6 Công cụ hỗ trợ kiểm thử hiệu năng.....	135
6.7 Công cụ hỗ trợ kiểm thử bảo mật	135
 CHƯƠNG 7: CÁC TIÊU CHUẨN TRONG QUẢN LÝ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM..	139
7.1 Giới thiệu	139
7.2 Đảm bảo chất lượng phần mềm trong các chuẩn của ISO.....	139
7.3 Đảm bảo chất lượng phần mềm trong các chuẩn CMM, CMMI	140
7.4 Cấu trúc và các level của CMMI :	144
7.4.1 Cấu trúc của CMMI :	144
7.4.2 Các level của CMMI:	144
7.4.3 Việt Nam áp dụng CMM/CMMI trong lĩnh vực phần mềm.	152
 TÀI LIỆU THAM KHẢO.....	153
 PHỤ LỤC	154
Phụ lục 1: Một số lỗi thường gặp trong phát triển phần mềm.....	154
Phụ lục 2: Một số hướng dẫn cho các loại kiểm thử.....	163
Phụ lục 3: Test plan mẫu.....	176

Chương 1: Giới thiệu đảm bảo chất lượng phần mềm

1.1 Khái niệm phần mềm

Trước khi tìm hiểu về đảm bảo về chất lượng phần mềm, mục này sẽ giới thiệu về khái niệm phần mềm.

Định nghĩa: Phần mềm bao gồm những thành phần sau đây:

- Chương trình máy tính
- Các thủ tục
- Tài liệu liên quan
- Dữ liệu cần thiết cho sự vận hành của hệ thống

Mỗi thành phần phần mềm đều có chức năng riêng và chất lượng của chúng đóng góp vào chất lượng chung của phần mềm và bảo trì phần mềm như sau:

- Chương trình máy tính được cần thiết là hiển nhiên vì chúng giúp máy tính vận hành thực thi các yêu cầu ứng dụng.
- Những thủ tục được yêu cầu để định nghĩa theo một thứ tự và lịch biểu của một chương trình khi thực thi, phương thức được triển khai và người chịu trách nhiệm cho thực thi các hoạt động cần thiết cho việc tác động vào phần mềm
- Nhiều kiểu tài liệu là cần thiết cho người phát triển, người sử dụng và người có nhiệm vụ duy trì. Tài liệu phát triển (báo cáo yêu cầu, báo cáo thiết kế, mô tả chương trình, v.v) cho phép sự phối hợp và cộng tác hiệu quả giữa các thành viên trong đội ngũ phát triển và hiệu quả trong việc xem lại và rà soát cả sản phẩm lập trình và thiết kế. Tài liệu sử dụng (thường là hướng dẫn sử dụng) cung cấp một sự miêu tả cho ứng dụng sẵn sàng và những phương pháp thích hợp cho họ sử dụng. Tài liệu bảo trì (tài liệu cho người phát triển) cung cấp cho đội bảo trì tất cả những thông tin yêu cầu về mã nguồn và công việc và cấu trúc cho từng module. Thông tin này được sử dụng để tìm nguyên nhân lỗi (bugs) hoặc thay đổi hoặc bổ sung thêm vào phần mềm có sẵn.
- Dữ liệu bao gồm các tham số đầu vào, mã nguồn và danh sách tên thích hợp với phần mềm để đặc tả những cái cần thiết cho người sử dụng thao tác với hệ thống. Một kiểu khác của dữ liệu cần thiết là chuẩn dữ liệu test, sử dụng để sách định rõ những thứ thay đổi không mong muốn trong mã nguồn

hoặc dữ liệu phần mềm đã từng xảy ra và những loại sự cố phần mềm nào có thể được lường trước.

1.2 Các nguyên nhân gây ra lỗi phần mềm

1.2.1 Một số ví dụ điển hình về lỗi phần mềm

Có rất nhiều trường hợp lỗi phần mềm đã gây thiệt hại hàng triệu đô la cũng như thiệt hại về người. Để thấy được mức độ nghiêm trọng cũng như sự đa dạng của lỗi phần mềm, mục này sẽ giới thiệu 10 lỗi nổi tiếng trong ngành phần mềm.

1. Ariane 5 Crash



Hình 1-0-1: Vụ nổ Ariane 5 do lỗi tràn số khi tính toán

Arian 5 là chiếc thứ năm trong loạt tàu vũ trụ dân dụng Ariane của châu Âu dùng để phóng vệ tinh vào không gian. Vào ngày 4 tháng 6 năm 1996 ở Kourou, Guiana của Pháp, chiếc Ariane 5 không người lái đã phát nổ chỉ khoảng 40 giây sau khi phóng. Chiếc tên lửa trị giá 500 triệu đô la này đã phát nổ do một lỗi phần mềm phổ biến được biết đến dưới tên gọi *Integer Overflow*. Lỗi này đã xảy ra trong quá trình thực hiện chuyển đổi dữ liệu từ số floating point 64-bit sang giá trị số integer 16-bit. Số floating point đã được chuyển đổi có giá trị lớn hơn số có thể được biểu diễn bởi một số integer 16-bit.

2. Lỗi phần mềm tên lửa Patriot



Hình 1-0-2: Hệ thống lá chắn tên lửa phán đoán sai vị trí do lỗi làm tròn số

Ngày 25 tháng 2 năm 1991 trong Chiến tranh vùng Vịnh, hệ thống tên lửa Patriot bỗng dưng đã không theo dõi và đánh chặn một tên lửa Scud đang tấn công một doanh trại của Mỹ. Theo Cơ quan Trách nhiệm Giải trình Chính phủ Hoa Kỳ, phần mềm đã bị trễ và đã không theo dõi việc phóng tên lửa trong thời gian thực, do đó nó đã để tên lửa của Iraq có cơ hội vượt qua và phát nổ trước khi có thể làm bất cứ điều gì để ngăn chặn nó. Mỹ đã có 28 người thiệt mạng và 100 người bị thương sau sự cố này.

3. Lỗi Y2K

Y2K là viết tắt của Year 2000 và được gọi là “*lỗi thiên niên kỷ*”. Vào cuối những năm 90, lỗi Y2K là lỗi được đề cập nhiều nhất khi cả thế giới chờ đợi máy bay va chạm, tàu vũ trụ biến mất, thị trường chứng khoán sụp đổ như dự đoán của nhiều chuyên gia công nghệ. Lỗi này là một sai lầm đơn giản trong hệ thống quản lý thời gian của các máy tính chỉ sử dụng hai chữ số để biểu thị một năm. Theo đó, 1970 sẽ được biểu diễn là 70 và năm 1999 sẽ được biểu diễn là 99. Lý do của việc này là để tiết kiệm bộ nhớ.

Khi gần đến năm 2000, các lập trình viên máy tính nhận ra rằng máy tính sẽ không thể biểu diễn chính xác năm 2000, vì 00 được dùng để biểu diễn năm 1900. Các hoạt động được lập trình hàng ngày hoặc hàng năm sẽ bị hư hỏng hoặc thiếu sót. Vào ngày 31 tháng 12 năm 1999, chuyển sang ngày 1 tháng 1 năm 2000, máy tính sẽ hiểu là từ ngày 31 tháng 12 năm 1999, chuyển sang ngày 1 tháng 1 năm 1900.

Các ngân hàng, tính lãi suất hàng ngày, phải đối mặt với những vấn đề thực sự. Lãi suất là số tiền mà người cho vay, ví dụ như ngân hàng, tính phí một khách hàng, chẳng hạn như một cá nhân hoặc một doanh nghiệp khi họ vay tiền. Thay vì tỷ lệ lãi suất cho một ngày, máy tính sẽ tính tỷ lệ lãi suất cho gần 100 năm !

Các trung tâm công nghệ, như các nhà máy điện, cũng bị đe dọa bởi lỗi Y2K. Nhà máy điện phụ thuộc máy tính để kiểm tra an toàn và bảo trì, chẳng hạn như áp lực nước hoặc mức độ bức xạ. Không có ngày chính xác sẽ làm mất những tính toán này và có thể đưa các cư dân gần đó đối mặt với nguy hiểm.

Giao thông vận tải cũng phụ thuộc vào thời gian và ngày tháng chính xác. Các hãng hàng không đặc biệt bị đe dọa vì máy tính lưu thông tin về tất cả các chuyến bay theo lịch trình sẽ bị đảo lộn hết cả.

Cuối cùng, có Y2K đã không gây ra hậu quả gì nghiêm trọng nhưng cũng phải mất một thời gian để các nhà phát triển phần mềm khắc phục triệt để lỗi này.

4. Khoản tiền gửi 92 nghìn triệu triệu đô la trên PayPal

Vào một ngày trong tháng 6 năm 2013, *Chris Reynolds* đã cảm thấy giật mình hoảng hốt trước khoản tiền có trong tài khoản PayPal của mình. Số dư tài khoản của nhân viên của *PR Pennsylvania* này đã tăng lên thành 92.233.720.368.547.800 USD .

Số tiền này đã được ghi có vào tài khoản PayPal của *Reynolds* do một lỗi phần mềm và làm anh trở thành người giàu nhất thế giới. PayPal thừa nhận rằng việc đó là do lỗi phần mềm của họ và PayPal đã đề nghị tặng một khoản tiền (không được công bố) cho *Reynolds*.

5. YouTube phải nâng cấp bộ đếm vì Gangnam Style

Năm 2014, YouTube bị lỗi bởi một video âm nhạc được gọi là Gangnam Style của *Psy*. Các nhà phát triển đã xây dựng YouTube trên nền tảng 32-bit, có nghĩa là YouTube có thể lưu và hiển thị số lượt xem của mỗi video bằng một con số nằm trong dải từ - 2,147,483,648 đến 2,147,483,647. Tức là số lượt xem lớn nhất có thể biểu diễn được trên Youtube là khoảng 2.15 tỷ và Youtube nghĩ rằng khó có video nào có thể đạt được lượt xem kinh khủng như vậy. Tuy nhiên video Gangnam Style đã phá vỡ bộ đếm lượt xem của YouTube khi vượt qua con số 2.147.483.647. (có lẽ do các bố, các mẹ, các bà liên tục cho con, cho cháu xem để dụ chúng nó ăn cháo, ăn cơm nên số lượt xem mới khủng như vậy).

“Chúng tôi không bao giờ nghĩ rằng sẽ có video được xem với số lượng lớn hơn một số nguyên 32-bit” YouTube cho biết trong một bài đăng trên Google +.



Hình 1-0-3: lỗi tràn số do số lượt xem bài Gangnam style quá lớn

Google sau đó đã sửa lỗi YouTube này bằng cách sử dụng một số nguyên 64-bit để lưu số lượt xem của mỗi video.

6. Lỗi tính toán của Windows Calculator

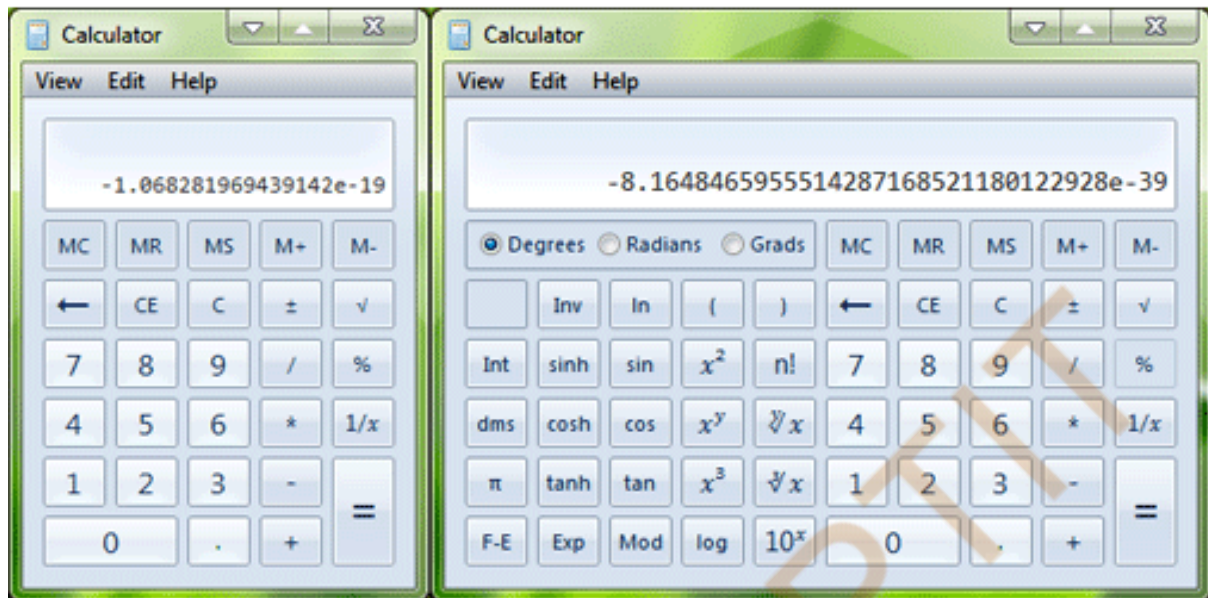
Đây là một lỗi trong Calculator của Windows, tồn tại ở các phiên bản Windows XP, Windows Vista, Windows 7 Windows 8. Lỗi xảy ra khi ta tính biểu thức: $\sqrt{4} - 2$ (lấy căn bậc hai của 4 rồi trừ đi 2). Câu trả lời đáng ra phải là 0 nhưng máy tính của Windows không cho ra kết quả 0.

Ở chế độ *Standard*, kết quả sẽ là $-1.068281969439142e-19$ (hình bên dưới)

Ở chế độ *Scientific*, kết quả sẽ là $8.1648465955514287168521180122928e-39$ (hình bên dưới)

Standard Mode

Scientific Mode



Hình 1-0-4: Lỗi tính toán sai trên Calculator

Điều tương tự cũng xảy ra với các số khác như: $\sqrt{9}$ -3, $\sqrt{16}$ -4, ...

Tuy nhiên trên Windows 10 thì lỗi này đã được fix.

7. Year 2038

Year 2038 là một vấn đề lỗi thời gian tương tự như vấn đề của Y2K chúng ta đã đề cập ở trên. Các máy tính chạy các hệ điều hành 32-bit trên thế giới có thể dừng lại vào ngày 19 tháng 1 năm 2038. Vấn đề năm 2038 là một vấn đề về tính toán và lưu trữ dữ liệu, trong đó các giá trị thời gian được tính và lưu trữ như một số nguyên 32-bit có dấu và số này được hiểu là số giây kể từ 00:00:00 giờ UTC vào ngày 1 tháng 1 năm 1970. Điều này dẫn đến việc sẽ không thể mã hóa thời gian sau thời điểm 03:14:07 UTC ngày 19 tháng 1 năm 2038, bởi vì sau thời điểm đó số giây sẽ lớn hơn $2^{31}-1 = 2,147,483,647$ giây và không thể được biểu diễn chính xác trong 32-bit và dẫn đến lỗi *integer overflow*.

Hiện nay vẫn chưa có giải pháp chính thức được đưa ra cho vấn đề Year 2038. Hy vọng đến năm 2038 thì tất cả các máy tính không còn dùng 32-bit.

8. Lỗi “Race Condition” trên phần mềm làm mất điện ảnh hưởng đến 50 triệu người

Vào ngày 14 tháng 8 năm 2003, việc mất điện trên 8 tiểu bang Hoa Kỳ và Canada đã ảnh hưởng tới 50 triệu người. Cơ quan *PC Authority* đã công bố nguyên nhân, đó là một lỗi phần mềm được biết đến với thuật ngữ “*race condition*” – một lỗi xảy ra khi hai *thread* riêng biệt của cùng một chương trình sử dụng cùng tài nguyên mà không có xử lý đồng bộ đúng cách dẫn đến sụp đổ hệ thống.

Đó là những gì đã xảy ra với kết quả là 256 nhà máy điện không hoạt động, gây ra sự gián đoạn lớn về truyền thông di động.

9. Tàu vũ trụ Mars Climate Orbiter trị giá 327 triệu đô la nổ tung do lỗi phần mềm

Tàu vũ trụ Mars Climate Orbiter (dự án 327,6 triệu đô la Mỹ) đã được NASA phóng vào ngày 11 tháng 12 năm 1998 để săn tìm các hành tinh có thể hỗ trợ cuộc sống con người. Thật không may, do lỗi trong phần mềm máy tính trên mặt đất, tàu Orbiter đã nổ tung sau 286 ngày. Do tính toán sai lầm, tàu Orbiter đã đi vào vào bầu khí quyển của sao Hỏa nhưng vào không đúng chỗ dẫn đến bị nổ tung ngay sau đó.

10. Nhà mạng AT&T của Mỹ gián đoạn 10 giờ đồng hồ vì lỗi phần mềm

Vào ngày 15 tháng 1 năm 1990, các khách hàng của nhà mạng AT&T không thể thực hiện được các cuộc gọi đường dài trong 9 giờ đồng hồ. Điều này làm tê liệt toàn bộ mạng điện thoại Hoa Kỳ. Tình trạng này xảy ra do một lỗi trong phần mềm kiểm soát các công tắc chuyển tiếp đường dài của AT&T. Khi đó, AT&T vừa trải qua một đợt cài đặt phần mềm mới, và phần mềm mới đã gây ra sự cố đáng tiếc. AT&T đã phải cài đặt lại phiên bản phần mềm cũ hơn nhưng đã tổn thất 60 triệu đô la cùng với sự tức giận của hàng triệu khách hàng.

1.2.2 Lỗi phần mềm

Có nhiều nguyên nhân gây ra lỗi phần mềm, biểu hiện của các lỗi cũng khác nhau ở mỗi giai đoạn phát triển phần mềm. Có ba loại lỗi phần mềm chính :

- **Error:** Là các phần của code mà không đúng một phần hoặc toàn bộ như là kết quả của lỗi ngữ pháp, logic hoặc lỗi khác được sinh ra bởi các nhà phân tích hệ thống, một lập trình viên hoặc các thành viên khác của đội phát triển phần mềm.
- **Fault:** Là các errors mà nó gây ra hoạt động không chính xác của phần mềm trong một ứng dụng cụ thể.
- **Failures:** Các faults trở thành failures chỉ khi chúng được “activated” đó là khi người dùng cố gắng áp dụng các phần mềm cụ thể đó bị faulty. Do đó, nguồn gốc của bất kì failure nào là một errors.

Ta hãy xem một ví dụ về bác sỹ chẩn đoán cho bệnh nhân. Bệnh nhân tới phòng khám với một danh sách failures (đó là các triệu chứng). Bác sỹ phải phát hiện ra các fault, hoặc nguồn gốc của các triệu chứng. Để trợ giúp quá trình chẩn đoán, bác sỹ có thể phải yêu cầu thực hiện các kiểm tra về sự bất thường của các trạng thái bên trong, như cao huyết áp, nhịp tim bất thường, lượng đường trong máu cao, cholesterol cao. Với thuật ngữ của chúng ta, các bất thường này gọi là errors.

Ta hãy xem xét một ví dụ về code Java sau:

```
public static int numZero (int[] x)
{ // Effects: if x == null throw NullPointerException
  // else return the number of occurrences of 0 in x
```

```

int count = 0;
for (int i = 1; i < x.length; i++) {
    if (x[i] == 0) {
        count++; }
}
return count;
}

```

fault ở đây là chương trình bắt đầu tìm kiếm các số bằng 0 từ chỉ số 1 thay vì chỉ số 0. Ví dụ numZero([2,7,0]) được tính chính xác là 1, còn với numZero([0,7,2]) sẽ được tính sai là 0. Trong cả 2 trường hợp, fault được thực thi. Mặc dù cả 2 trường hợp đều dẫn tới 1 lỗi, nhưng chỉ có trường hợp thứ 2 là gây ra failure. Để hiểu về trạng thái lỗi, ta cần định nghĩa trạng thái của chương trình. Trạng thái của numZero bao gồm giá trị của biến x, count, i và program counter (PC). Với ví dụ đầu tiên, trạng thái ở câu lệnh if cho vòng lặp đầu tiên là (x = [2, 7, 0], count = 0, i = 1, PC = if). Lưu ý là trạng thái này là lỗi vì đáng ra i=0. Tuy nhiên, trạng thái lỗi này không được lan truyền tới output và do vậy phần mềm không fail. Nói cách khác, một trạng thái là lỗi nếu nó không phải là trạng thái như mong đợi dù cho tất cả các giá trị trong trạng thái là chấp nhận được. Tổng quát hơn, nếu chuỗi trạng thái mong đợi là s0,s1,s2... trong khi chuỗi trạng thái thực tế là s0,s2,s3... thì trạng thái s2 là lỗi. Với trường hợp thứ 2, trạng thái tương ứng là (x = [0, 7, 2], count = 0, i = 1, PC = if). Trong trường hợp này, lỗi lan truyền tới kết quả trả về. Do vậy, ta thu được kết quả failure.

Định nghĩa về fault và failure cho phép ta phân biệt giữa testing và debugging.

1.2.3 Nguyên nhân gây ra lỗi phần mềm

Việc phát hiện ra lỗi là cần thiết, nhưng tìm ra nguyên nhân gây lỗi để tránh lỗi trong tương lai mới thực sự quan trọng. Chín nguyên nhân gây ra lỗi phần mềm thống kê sau đây đã được tổng kết sau nhiều năm nghiên cứu :

1. Định nghĩa yêu cầu lỗi
2. Lỗi giao tiếp giữa khách hàng và người phát triển
3. Sự thiếu rõ ràng của các yêu cầu phần mềm
4. Lỗi thiết kế logic
5. Lỗi coding
6. Không phù hợp với tài liệu và chỉ thị coding
7. Thiếu sót trong quá trình kiểm thử

8. Lỗi thủ tục

9. Lỗi tài liệu

Nội dung cụ thể mỗi nguyên nhân được xác định như sau:

1.2.3.1 Định nghĩa các yêu cầu bị lỗi

Việc xác định các lỗi yêu cầu, thường do khách hàng, là một trong những nguyên nhân chính của các lỗi phần mềm. Các lỗi phổ biến nhất loại này là:

- Sai sót trong định nghĩa các yêu cầu.
- Không có các yêu cầu quan trọng.
- Không hoàn chỉnh định nghĩa các yêu cầu.
- Bao gồm các yêu cầu không cần thiết, các chức năng mà không thực sự cần thiết trong tương lai gần.

1.2.3.2 Các lỗi trong giao tiếp giữa khách hàng và nhà phát triển

- Hiểu lầm trong giao tiếp giữa khách hàng và nhà phát triển là nguyên nhân bổ sung cho các lỗi ưu tiên áp dụng trong giai đoạn đầu của quá trình phát triển: Hiểu sai các chỉ dẫn của khách hàng như đã nêu trong các tài liệu yêu cầu. Hiểu sai các yêu cầu thay đổi của khách hàng được trình bày với nhà phát triển bằng văn bản trong giai đoạn phát triển.
- Hiểu sai của các yêu cầu thay đổi của khách hàng được trình bày bằng lời nói với nhà phát triển trong giai đoạn phát triển.
- Hiểu sai về phản ứng của khách hàng đối với các vấn đề thiết kế trình bày của nhà phát triển.

Thiếu quan tâm đến các đề nghị của khách hàng đề cập đến yêu cầu thay đổi và khách hàng trả lời cho các câu hỏi nêu ra bởi nhà phát triển trên một phần của nhà phát triển.

1.2.3.3 Sai lệch có chủ ý từ các yêu cầu phần mềm

Trong một số trường hợp, các nhà phát triển có thể cố tình đi chệch khỏi các yêu cầu trong tài liệu, hành động thường gây ra lỗi phần mềm. Các lỗi trong những trường hợp này là sản phẩm phụ của các thay đổi. Các tình huống thường gặp nhất là: Phát triển các module phần mềm Các thành phần sử dụng lại lấy từ một dự án trước đó mà không cần phân tích đầy đủ về những thay đổi và thích nghi cần thiết để thực hiện một cách chính xác tất cả các yêu cầu mới.

Do thời gian hay áp lực ngân sách, nhà phát triển quyết định bỏ qua một phần của các yêu cầu các chức năng trong một nỗ lực để đối phó với những áp lực này. Nhà phát triển-khởi xướng, không được chấp thuận các cải tiến cho phần mềm, mà

không có sự chấp thuận của khách hàng, thường xuyên bỏ qua các yêu cầu có vẻ nhỏ đối với nhà phát triển. Như vậy những thay đổi "nhỏ" có thể, cuối cùng, gây ra lỗi phần mềm.

1.2.3.4 Các lỗi thiết kế logic

Lỗi phần mềm có thể đi vào hệ thống khi các chuyên gia thiết kế hệ thống-các kiến trúc sư hệ thống, kỹ sư phần mềm, các nhà phân tích, vv - Xây dựng phần mềm yêu cầu. Các lỗi điển hình bao gồm:

- + Định nghĩa các yêu cầu phần mềm bằng các thuật toán sai lầm.
- + Quy trình định nghĩa có chứa trình tự lỗi.
- + Sai sót trong các định nghĩa biên
- + Thiếu sót trong các trạng thái hệ thống phần mềm được yêu cầu
- + Thiếu sót trong định nghĩa các hoạt động trái pháp luật trong hệ thống phần mềm

1.2.3.5 Các lỗi coding

Một loạt các lý do các lập trình viên có thể gây ra các lỗi code. Những lý do này bao gồm sự hiểu lầm các tài liệu thiết kế, ngôn ngữ sai sót trong ngôn ngữ lập trình, sai sót trong việc áp dụng các CASE và các công cụ phát triển khác, sai sót trong lựa chọn dữ liệu...

1.2.3.6 Không tuân thủ theo các tài liệu hướng dẫn và mã hóa

Hầu hết các đơn vị phát triển có tài liệu hướng dẫn và tiêu chuẩn mã hóa riêng của mình để xác định nội dung, trình tự và định dạng của văn bản, và code tạo ra bởi các thành viên. Để hỗ trợ yêu cầu này, đơn vị phát triển và công khai các mẫu và hướng dẫn mã hóa. Các thành viên của nhóm phát triển, đơn vị được yêu cầu phải thực hiện theo các yêu cầu này.

1.2.3.7 Thiếu sót trong quá trình kiểm thử

Thiếu sót trong quá trình kiểm thử ảnh hưởng đến tỷ lệ lỗi bằng cách để lại một số lỗi lớn hơn không bị phát hiện hoặc không phát hiện đúng. Những kết quả yếu kém từ các nguyên nhân sau đây:

- Kế hoạch kiểm thử chưa hoàn chỉnh để lại phần không được điều chỉnh của phần mềm hoặc các chức năng ứng dụng và các trạng thái của hệ thống. Failures trong tài liệu và báo cáo phát hiện sai sót và lỗi lầm.
- Nếu không kịp thời phát hiện và sửa chữa lỗi phần mềm theo như hướng dẫn nhập nhằng cho lỗi này.
- Không hoàn chỉnh sửa chữa các lỗi được phát hiện do sơ suất hay giới hạn thời gian.

1.2.3.8 Các lỗi thủ tục

Các thủ tục trực tiếp cho người sử dụng đối với các hoạt động là cần thiết ở mỗi bước của quá trình. Chúng có tầm quan trọng đặc biệt trong các hệ thống phần mềm phức tạp, nơi các tiến trình được tiến hành một vài bước, mỗi bước trong số đó có thể có nhiều kiểu dữ liệu và cho phép kiểm tra các kết quả trung gian.

1.2.3.9 Các lỗi về tài liệu

Các lỗi về tài liệu là vấn đề của các đội phát triển và bảo trì đều có sai sót trong tài liệu thiết kế và trong tài liệu hướng dẫn tích hợp trong thân của phần mềm. Những lỗi này có thể là nguyên nhân gây ra lỗi bổ sung trong giai đoạn phát triển tiếp và trong thời gian bảo trì.

Cần nhấn mạnh rằng tất cả các nguyên nhân gây ra lỗi đều là con người, công việc của các nhà phân tích hệ thống, lập trình, kiểm thử phần mềm, các chuyên gia tài liệu, và thậm chí cả các khách hàng và đại diện của họ.

1.3 Đảm bảo chất lượng phần mềm

1.3.1 Khái niệm

Theo IEEE, chất lượng phần mềm được định nghĩa như sau :

Chất lượng phần mềm là:

- *Mức độ mà một hệ thống, thành phần hoặc một tiến trình đạt được yêu cầu đã đặc tả*
- *Mức độ mà một hệ thống, thành phần hoặc một tiến trình đạt được những nhu cầu hay mong đợi của khách hàng hoặc người sử dụng.*

Ban đầu đảm bảo chất lượng phần mềm có mục tiêu đạt được các yêu cầu đề ra, tuy nhiên thực tế phát triển phần mềm tồn tại rất nhiều ràng buộc đòi hỏi người phát triển cần tối ưu hóa công tác quản lý.

Khái niệm đảm bảo chất lượng phần mềm được xác định như sau :

Đảm bảo chất lượng phần mềm là một tập các hoạt động đã được lập kế hoạch và có hệ thống, cần thiết để cung cấp đầy đủ sự tin cậy vào quy trình phát triển phần mềm hay quy trình bảo trì phần mềm của sản phẩm hệ thống phần mềm phù hợp với các yêu cầu chức năng kỹ thuật cũng như với các yêu cầu quản lý mà giữ cho lịch biểu và hoạt động trong phạm vi ngân sách.

1.3.2 Mục tiêu đảm bảo chất lượng phần mềm

Phát triển phần mềm luôn đi đôi với bảo trì, vì vậy các hoạt động bảo đảm chất lượng phần mềm đều có mối liên quan chặt chẽ đến bảo trì. Những mục tiêu đảm bảo chất lượng phần mềm tương ứng với giai đoạn phát triển và bảo trì được xác định cụ thể như sau :

- Phát triển phần mềm (hướng tiến trình)

1. Đảm bảo một mức độ chấp nhận được rằng phần mềm sẽ thực hiện được các yêu cầu chức năng.
2. Đảm bảo một mức độ chấp nhận được rằng phần mềm sẽ đáp ứng được các yêu cầu về lịch biểu và ngân sách
3. Thiết lập và quản lý các hoạt động để cải thiện và nâng cao hiệu quả của phát triển phần mềm và các hoạt động SQA.

- Bảo trì phần mềm (hướng sản phẩm)

1. Đảm bảo một mức độ chấp nhận được rằng các hoạt động bảo trì phần mềm sẽ đáp ứng được các yêu cầu chức năng.
2. Đảm bảo một mức độ chấp nhận được rằng các hoạt động bảo trì phần mềm sẽ đáp ứng được các yêu cầu về lịch biểu và ngân sách
3. Thiết lập và quản lý các hoạt động để cải thiện và nâng cao hiệu quả của bảo trì phần mềm.

1.3.3 Xác minh, thẩm định và đánh giá chất lượng

Ba khía cạnh đảm bảo chất lượng của sản phẩm phần mềm gồm Verification, Validation, và Qualification.

- **Verification:** quá trình đánh giá một hệ thống hay một thành phần để xác định xem những sản phẩm của một pha phát triển xác định có thỏa mãn những điều kiện được đặt gia khi bắt đầu pha đó hay không.
- **Validation:** quá trình đánh giá một hệ thống hay một thành phần trong suốt hoặc khi kết thúc quá trình phát triển để xác định xem nó có thỏa mãn những yêu cầu đã đặc tả hay không.
- **Qualification:** quá trình xác định một hệ thống hoặc một thành phần có phù hợp với việc sử dụng hay không.

Theo những định nghĩa của IEEE, verification kiểm tra tính nhất quán của sản phẩm đang phát triển với những sản phẩm đã được phát triển ở pha trước. Khi thực hiện, người thẩm tra đi sau quy trình phát triển và giả sử rằng tất cả những pha phát triển

đăng trước đã được hoàn thành một cách chính xác hoặc là như kế hoạch gốc hoặc là sau khi đã sửa chữa những sai sót được phát hiện.

Validation miêu tả sự quan tâm của khách hàng, bằng cách thẩm tra những yêu cầu gốc của họ.

Qualification tập trung vào những khía cạnh hoạt động, ở đó việc bảo trì là vấn đề chính. Một thành phần phần mềm đã được phát triển và tài liệu hóa theo những chuẩn, kiểu, và cấu trúc chuyên nghiệp sẽ dễ dàng để bảo trì hơn những thành phần có code lộn xộn.

Người lên kế hoạch cần phải xác định xem những khía cạnh nào nên được sát hạch trong mỗi hoạt động đảm bảo chất lượng phần mềm.

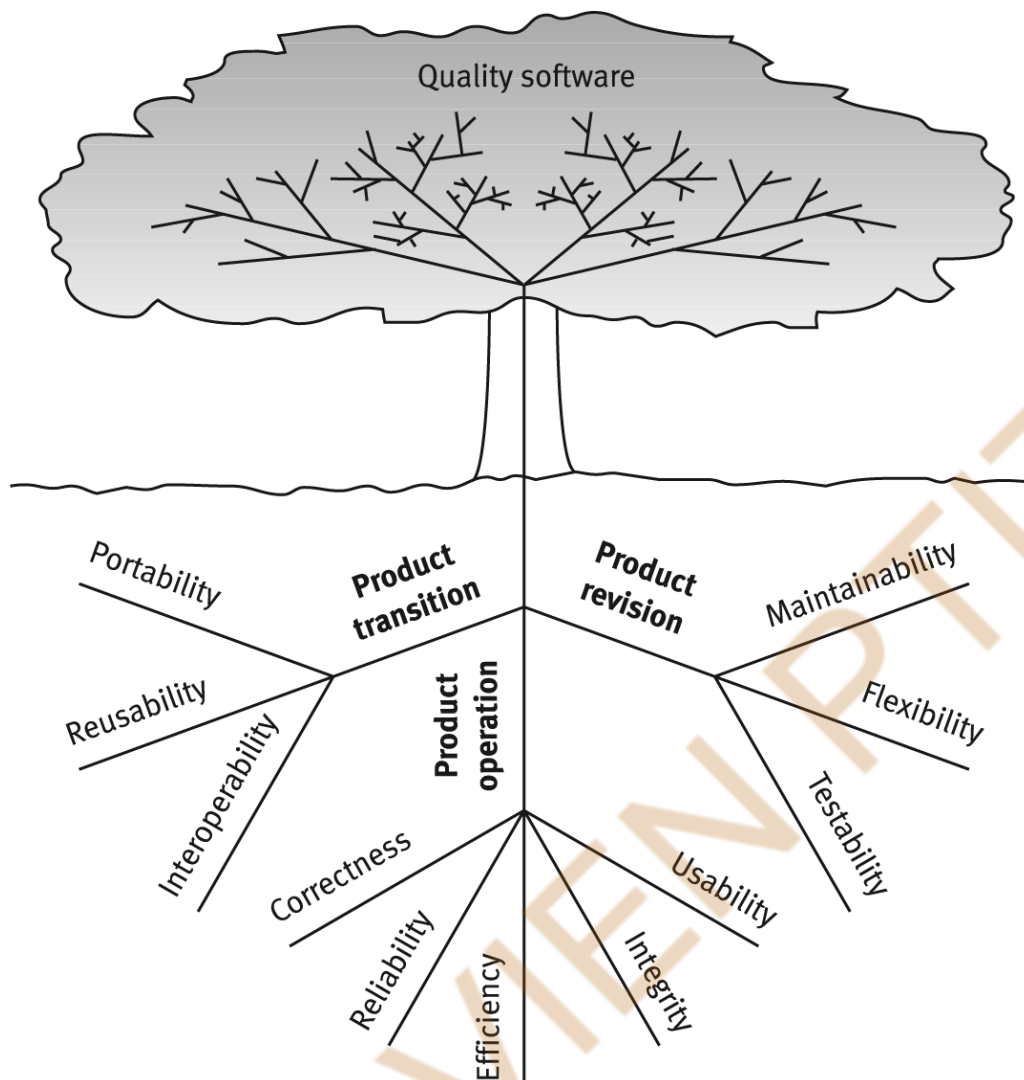
Xác minh thường là hoạt động có tính kỹ thuật cao hơn, sử dụng những tri thức về các yêu cầu, đặc tả phần mềm. Xác nhận thường phụ thuộc vào tri thức về lĩnh vực tương ứng. Cụ thể là, tri thức về ứng dụng của phần mềm được viết. Ví dụ, xác nhận của phần mềm về máy bay yêu cầu tri thức từ kỹ sư hàng không và phi công.

Cụm từ IV & V - independent verification and validation- nghĩa là xác nhận và xác minh độc lập. IV & V yêu cầu việc đánh giá phải được thực hiện bởi người không phải là lập trình viên. Một số trường hợp đội IV & V thuộc dự án, hoặc thuộc cùng công ty, cũng có thể thuộc một tổ chức độc lập. Vì sự độc lập của IV & V, tiến trình thường chưa bắt đầu cho tới khi dự án kết thúc và thường được thực hiện bởi chuyên gia trong lĩnh vực hơn là người phát triển phần mềm.

1.4 Các tiêu chí chất lượng

Đã có nhiều tác giả nghiên cứu về các yếu tố chất lượng phần mềm từ các yêu cầu của nó. Theo thời gian có thể quan niệm về việc đảm bảo chất lượng phần mềm có phần thay đổi, tuy nhiên mô hình các yếu tố đảm bảo chất lượng phần mềm của McCall ra đời vào những năm 70 của thế kỷ trước vẫn còn được nhiều người nhắc đến như là cơ sở tham chiếu các yêu cầu phần mềm. Sau McCall cũng có một số mô hình được quan tâm như mô hình do Evans, Marciniak hay mô hình của Deutsch và Willis, tuy nhiên những mô hình này chỉ bổ sung hay sửa đổi một vài yếu tố chất lượng. Theo McCall, các yếu tố chất lượng phần mềm được chia làm ba loại :

- Các yếu tố hoạt động của sản phẩm bao gồm tính chính xác, tin cậy, hiệu quả, tính toàn vẹn, sử dụng được
- Các yếu tố rà soát bao gồm tính bảo trì, linh hoạt, có thể test được
- Các yếu tố chuyển giao bao gồm tính khả chuyển, có khả năng sử dụng lại, có khả năng giao tác.



Hình 1-0-5: Cây mô hình tiêu chí chất lượng phần mềm theo McCall

Chi tiết các thuộc tính được phân tích như sau :

(1) **Các yếu tố vận hành sản phẩm** : Sự chính xác, độ tin cậy, tính hiệu quả, tính toàn vẹn và khả năng sử dụng được :

- **Sự chính xác** : Các yêu cầu về độ chính xác được xác định trong một danh sách các đầu ra cần thiết của hệ thống phần mềm, như màn hình hiển thị truy vấn số dư của khách hàng trong một hệ thống thông tin kế toán bán hàng... Các đặc tả đầu ra thường là đa chiều, một số chiều thông dụng là :

- Nhiệm vụ đầu ra (ví dụ : bản in hóa đơn bán hàng, hay đèn báo động đỏ khi nhiệt độ tăng lên trên 250 độ F)
- Độ chính xác yêu cầu của các đầu ra này; chúng có thể bị ảnh hưởng bất lợi bởi các tính toán không chính xác hay các dữ liệu không chính xác.

- Tính đầy đủ của thông tin đầu ra; chúng có thể bị ảnh hưởng bất lợi bởi dữ liệu không đầy đủ.
- Up-to-dateness của thông tin (xác định bằng thời gian giữa sự kiện và việc xem xét hệ thống phần mềm.
- Độ sẵn sàng của thông tin (thời gian đáp ứng : được định nghĩa là thời gian cần thiết để có được các thông tin yêu cầu)
- Các chuẩn cho việc code và viết tài liệu cho hệ thống phần mềm.
- **Độ tin cậy :** Các yêu cầu về độ tin cậy giải quyết các lỗi để cung cấp dịch vụ. Chúng xác định tỷ lệ lỗi hệ thống phần mềm tối đa cho phép, các lỗi này có thể là lỗi toàn bộ hệ thống hoặc một hay nhiều chức năng riêng biệt của nó.
- **Tính hiệu quả :** Các yêu cầu về tính hiệu quả giải quyết vấn đề về các tài nguyên phần cứng cần thiết để thực hiện tất cả các chức năng của hệ thống phần mềm với sự phù hợp của tất cả các yêu cầu khác. Các tài nguyên phần cứng chính được xem xét ở đây là khả năng xử lý của máy tính (được đo bằng MIPS – triệu lệnh/giây; MHz – triệu chu kỳ/giây...); khả năng lưu trữ dữ liệu (dung lượng bộ nhớ, dung lượng đĩa – được đo bằng MBs, GBs, TBs...) và khả năng truyền dữ liệu (thường được đo bằng MBPS, GBPS). Các yêu cầu này có thể bao gồm cả các giá trị tối đa tài nguyên phần cứng được sử dụng trong hệ thống phần mềm. Một yêu cầu khác về tính hiệu quả đó là thời gian giữa các lần phải sạc điện đối với các hệ thống nằm trên các máy tính xách tay hay các thiết bị di động.
- **Tính toàn vẹn:** các yêu cầu về tính toàn vẹn giải quyết các vấn đề về bảo mật hệ thống phần mềm, các yêu cầu này để ngăn chặn sự truy cập trái phép, để phân biệt giữa phần lớn nhân viên chỉ được phép xem thông tin với một nhóm hạn chế những người được phép thêm và thay đổi dữ liệu...
- **Khả năng sử dụng:** Các yêu cầu về khả năng sử dụng được sẽ đưa ra phạm vi của tài nguyên nhân lực cần thiết để đào tạo một nhân viên mới và để vận hành hệ thống phần mềm.
-

(2) **Các yếu tố về rà soát sản phẩm :** bảo trì được, linh động và kiểm tra được :

- **Khả năng bảo trì được :** Các yêu cầu về khả năng bảo trì được sẽ xác định người dùng và nhân viên bảo trì phải nỗ lực thế nào để xác định được nguyên nhân của các lỗi phần mềm, để sửa lỗi và để xác nhận việc sửa lỗi thành công. Các yêu cầu của yếu tố này nói tới cấu trúc modul của phần mềm, tài liệu chương trình nội bộ và hướng dẫn sử dụng của lập trình viên...

- **Tính linh động** : Các yêu cầu về tính linh động cũng bao gồm cả các khả năng và nỗ lực cần thiết để hỗ trợ các hoạt động bảo trì. Chúng gồm các nguồn lực (man-day) cần thiết để thích nghi với một gói phần mềm, với các khách hàng trong cùng nghề, với các mức độ hoạt động khác nhau, với các loại sản phẩm khác nhau... Các yêu cầu về yếu tố này cũng hỗ trợ các hoạt động bảo trì trở nên hoàn hảo, như thay đổi và bổ sung vào phần mềm để tăng dịch vụ của nó và để thích nghi với các thay đổi trong môi trường thương mại và kỹ thuật của công ty.

- **Khả năng test được** : Các yêu cầu về khả năng kiểm tra được nói tới việc kiểm tra sự vận hành có tốt hay không của các hệ thống thông tin. Các yêu cầu về khả năng kiểm tra được liên quan tới các tính năng đặc biệt trong chương trình giúp người tester dễ dàng thực hiện công việc của mình hơn, ví dụ như đưa ra các kết quả trung gian. Các yêu cầu về khả năng kiểm tra được liên quan tới vận hành phần mềm bao gồm các chuẩn đoán tự động được thực hiện bởi hệ thống phần mềm trước khi bắt đầu hệ thống, để tìm hiểu xem có phải tất cả các thành phần của hệ thống phần mềm đều làm việc tốt hay không, và để có một bản báo cáo về các lỗi đã được phát hiện. Một loại khác của yêu cầu này là việc check các dự đoán tự động, được các kỹ thuật viên bảo trì sử dụng để phát hiện nguyên nhân gây lỗi phần mềm.

(3) **Các yếu tố về chuyển giao sản phẩm** : tính lưu động (khả năng thích nghi với môi trường), khả năng tái sử dụng và khả năng cộng tác được :

- **Tính lưu động** : các yêu cầu về tính lưu động nói tới khả năng thích nghi của hệ thống phần mềm với các môi trường khác, bao gồm phần cứng khác, các hệ điều hành khác... Các yêu cầu này đòi hỏi các phần mềm cơ bản có thể tiếp tục sử dụng độc lập hoặc đồng thời trong các trường hợp đa dạng.

- **Khả năng tái sử dụng** : Các yêu cầu về khả năng tái sử dụng nói tới việc sử dụng các modul phần mềm trong một dự án mới đang được phát triển mà các modul này ban đầu được thiết kế cho một dự án khác. Các yêu cầu này cũng cho phép các dự án tương lai có thể sử dụng một modul đã có hoặc một nhóm các modul hiện đang được phát triển. Tái sử dụng phần mềm sẽ tiết kiệm tài nguyên phát triển, rút ngắn thời gian phát triển và tạo ra các moduls chất lượng cao hơn. Chất lượng modul cao hơn là dựa trên giả định rằng hầu hết các lỗi phần mềm đều được phát hiện bởi các hoạt động đảm bảo chất lượng phần mềm thực hiện trên phần mềm ban đầu, bởi những người sử dụng phần mềm ban đầu và trong suốt những lần tái sử dụng trước của nó. Các vấn đề về tái sử dụng phần mềm đã trở thành một phần trong chuẩn công nghiệp phần mềm (IEEE, 1999).

- **Khả năng cộng tác** : Các yêu cầu về khả năng cộng tác tập trung vào việc tạo ra các giao diện với các hệ thống phần mềm khác. Các yêu cầu về khả năng cộng tác có thể xác định tên của phần mềm với giao diện bắt buộc. Chúng cũng có thể xác định cấu trúc đầu ra được chấp nhận như một tiêu chuẩn trong một ngành công nghiệp cụ thể hoặc một lĩnh vực ứng dụng.

1.5 Các tiêu chí chất lượng ảnh hưởng tới hoạt động đảm bảo chất lượng phần mềm như nào.

Những hoạt động đảm bảo chất lượng là hướng quy trình, nói cách khác, có liên kết tới sự hoàn thành của một pha dự án, sự hoàn tất một mốc dự án, và nhiều hơn nữa. Những hoạt động đảm bảo chất lượng được tích hợp vào trong kế hoạch phát triển.

Những người lên kế hoạch đảm bảo chất lượng cho một dự án cần xác định:

- Danh sách những hoạt động đảm bảo chất lượng cần thiết cho dự án.
- Với mỗi hoạt động đảm bảo chất lượng:
 - Thời gian.
 - Loại hoạt động đảm bảo chất lượng áp dụng.
 - Người thực hiện hoạt động và tài nguyên yêu cầu.
 - Những tài nguyên yêu cầu cho việc khắc phục những nhược sai sót và những thay đổi.

Cường độ của những hoạt động đảm bảo chất lượng đã được lên kế hoạch được chỉ ra bởi số những hoạt động yêu cầu. Những yếu tố dự án và nhóm ảnh hưởng tới cường độ như sau:

Yếu tố dự án:

- Độ lớn của dự án.
- Sự phức tạp và khó của kỹ thuật.
- Phạm vi của những thành phần sử dụng lại.
- Hậu quả nếu dự án bị lỗi.

Yếu tố nhóm:

- Trình độ chuyên môn của những thành viên trong nhóm.
- Sự quen thuộc của nhóm với dự án và Kinh nghiệm của nhóm trong lĩnh vực.
- Tính sẵn sàng của những thành viên có thể trợ giúp nhóm.

- Sự hiểu biết giữa những thành viên trong nhóm, hay nói cách khác là số thành viên mới trong nhóm.

THU VIEN PTIT

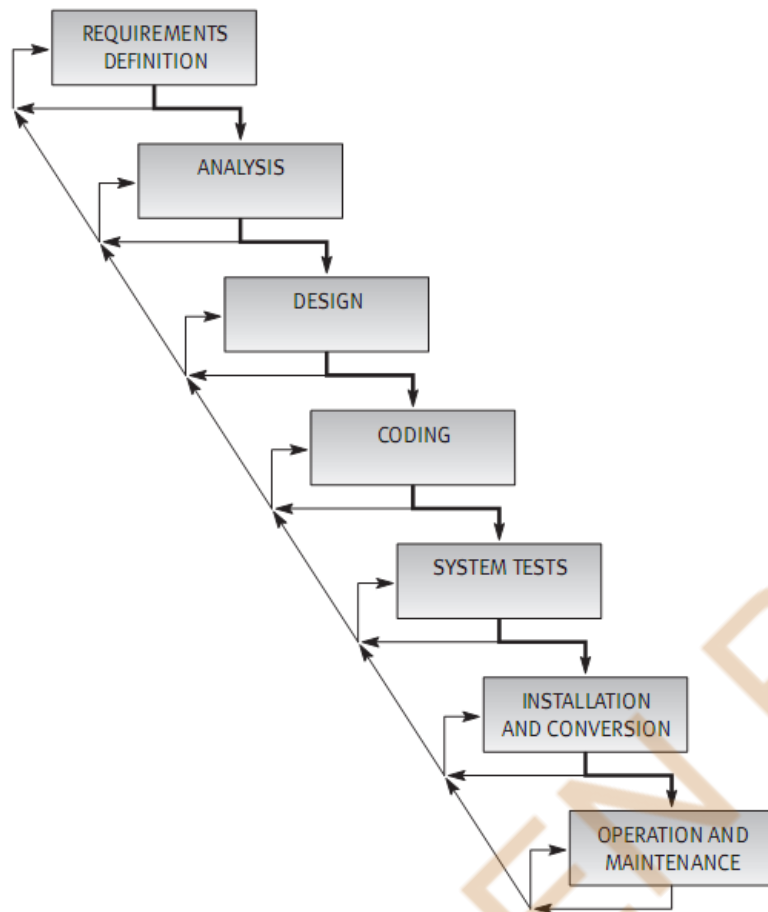
Chương 2: Tích hợp các hoạt động đảm bảo chất lượng phần mềm vào vòng đời phát triển phần mềm

2.1 Các phương pháp phát triển phần mềm

- Mô hình vòng đời phát triển phần mềm (SDLC)

Mô hình vòng đời phát triển phần mềm (SDLC) là một mô hình truyền thống (hiện nay vẫn có thể áp dụng được); nó cung cấp sự mô tả toàn diện nhất về quy trình phát triển phần mềm. Mô hình chỉ ra cần xây dựng những khối chính cho toàn bộ quá trình phát triển, được mô tả như là một chuỗi tuyến tính. Trong pha ban đầu của quy trình phát triển phần mềm, các tài liệu thiết kế sản phẩm được chuẩn bị, việc đánh giá phiên bản đầu tiên của chương trình máy tính chỉ diễn ra ở giai đoạn cuối của quy trình. Mô hình SDLC có thể đóng vai trò như một khung (framework) để biểu diễn các mô hình khác.

Thể hiện phổ biến nhất của SDLC là mô hình waterfall (thác nước). Mô hình này được minh họa trong hình dưới đây:



Hình 2-1: Mô hình vòng đời phát triển phần mềm Thác nước

Mô hình gồm có 7 pha: *xác định yêu cầu, phân tích, thiết kế, coding, kiểm thử hệ thống, cài đặt và chuyển giao, vận hành và bảo trì.*

Xác định yêu cầu: Mục đích của pha xác định yêu cầu là xác định các chức năng của hệ thống cần xây dựng, khách hàng phải đưa ra và xác định các quyết định của họ. Trong nhiều trường hợp hệ thống phần mềm là một phần của hệ thống lớn hơn. Thông tin về các phần khác của hệ thống được mở rộng sẽ giúp thiết lập sự cộng tác giữa đội dự án và các giao diện thành phần phát triển.

Phân tích: Nỗ lực chính ở đây là phân tích sự ảnh hưởng của những yêu cầu thành mô hình khởi tạo của hệ thống phần mềm.

Thiết kế: Giai đoạn này bao gồm việc định nghĩa chi tiết đầu vào, đầu ra và các thủ tục xử lý, bao gồm cấu trúc dữ liệu, cơ sở dữ liệu, cấu trúc phần mềm, ...

Coding: Trong pha này, toàn bộ thiết kế được chuyển thành code. Việc coding bao gồm những hoạt động đảm bảo chất lượng phần mềm như inspection, unit test, và test tích hợp.

Test hệ thống (System test): Test hệ thống được thực hiện sau khi pha coding hoàn thiện. Mục đích chính của việc kiểm thử là càng tìm ra nhiều lỗi phần mềm càng tốt để đạt được mức độ chấp nhận của chất lượng phần mềm. Kiểm thử hệ thống được thực hiện bởi những người phát triển trước khi bàn giao cho khách hàng. Trong nhiều trường hợp, khách hàng thực hiện kiểm thử phần mềm một cách độc lập (còn gọi là test chấp nhận sản phẩm) để đảm bảo rằng những người phát triển đã thỏa mãn tất cả những hứa hẹn và những phản ứng phần mềm bất ngờ hoặc lỗi có thể được thấy trước. Đây là điều khá phổ biến, khách hàng yêu cầu người phát triển để họ được tham gia vào trong quá trình kiểm thử hệ thống, một thủ tục đã tiết kiệm được về mặt thời gian và tài nguyên cho việc phân tách riêng biệt giữa test hệ thống và test chấp nhận.

Cài đặt và chuyển giao: Sau khi hệ thống phần mềm được phê chuẩn, hệ thống được cài đặt để phục vụ như là một phần sụn, có nghĩa là, nó như là một phần của hệ thống thông tin, biểu diễn một thành phần cơ bản của hệ thống được mở rộng. Nếu hệ thống thông tin mới được xây dựng để thay thế hệ thống đang tồn tại, một quy trình chuyển giao phải được khởi tạo để đảm bảo rằng các hoạt động của tổ chức vẫn được tiếp tục mà không bị gián đoạn trong suốt pha chuyển giao.

Vận hành và bảo trì: Vận hành phần mềm bắt đầu sau khi pha cài đặt và chuyển giao đã được xong. Xuyên suốt thời kỳ vận hành, thường ít nhất là một vài năm hoặc cho đến khi xuất hiện phần mềm mới, việc bảo trì là cần thiết. Việc bảo trì kết hợp ba kiểu dịch vụ: thích ứng – sử dụng các đặc trưng của phần mềm đang tồn tại để thực hiện các yêu cầu mới; hoàn thiện – thêm một số đặc trưng để cải thiện hiệu năng của phần mềm; sửa lỗi – sửa các lỗi phần mềm được tìm thấy bởi người sử dụng trong quá trình vận hành.

Số lượng các pha có thể thay đổi tùy theo đặc trưng của từng dự án. Trong các dự án phức tạp, các mô hình phần mềm cỡ lớn, một số pha có thể bị chia ra, do đó, số lượng các pha có thể lên đến tám, chín hoặc thậm chí là nhiều pha hơn nữa. Trong các dự án nhỏ, một số pha lại có thể bị gộp lại, giảm số lượng pha của SDLC xuống còn sáu, năm, thậm chí là chỉ còn bốn pha.

Ở cuối mỗi pha, các đầu ra được xem xét và đánh giá bởi người phát triển, và trong một số trường hợp cũng có thể là khách hàng tham gia. Các kết quả có thể của quá trình xem xét lại bao gồm:

- Sự phê chuẩn của các đầu ra ở pha đó và quy trình của pha tiếp theo
- Các yêu cầu chỉnh sửa, làm lại hoặc thay đổi các phần của pha cuối cùng; trong trường hợp đảm bảo tính chắc chắn, có thể trả về các pha gần nhất theo yêu cầu.

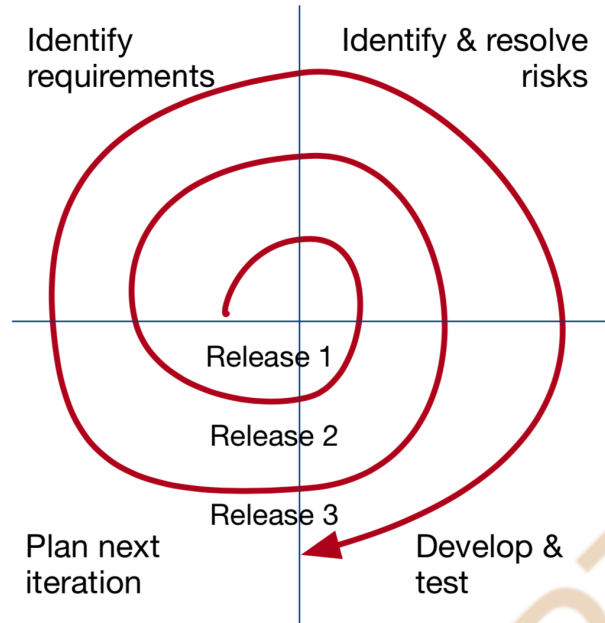
Độ rộng của đường kết nối giữa các hình hộp chữ nhật trong hình minh họa phản ánh khả năng có những kết quả khác nhau. Do đó, hầu hết các quy trình được thực hiện như

Mô hình Waterfall đã được đề xuất bởi Royce (1970) và sau đó đưa ra trong trong mô hình chung của nó được biết đến như là Boehm (1981). Mô hình này cung cấp các nền tảng cho phần lớn các chuẩn đảm bảo chất lượng phần mềm đã được triển khai, chẳng hạn như chuẩn IEEE 1012, IEEE 12207, ...

```
graph TD; RP[Rapid prototype] --> A[Analysis]; A --> D[Design]; D --> I[Implementation]; I --> PM[Postdelivery maintenance]; PM --> R[Retirement]; CR[Changed requirements] -.-> RP; CR -.-> A; CR -.-> D; CR -.-> I; CR -.-> PM;
```

The flowchart illustrates the SDLC process, starting with a 'Rapid prototype' box leading to 'Analysis', 'Design', 'Implementation', 'Postdelivery maintenance', and finally 'Retirement'. A dashed box labeled 'Changed requirements' is connected to all five main process boxes via dashed arrows, indicating feedback loops. A legend at the bottom left identifies solid arrows as 'Development' and dashed arrows as 'Maintenance'.

Mô hình xoắn ốc cung cấp một phương pháp luận nhằm đảm bảo hiệu quả về mặt hiệu năng của mỗi pha trong mô hình SDLC truyền thống. Mô hình này liên quan đến một quy trình lặp, tích hợp các yêu cầu thay đổi của khách hàng, phân tích và giải quyết các rủi ro, các hoạt động về mặt kỹ nghệ và kế hoạch cho phát triển phần mềm. Một trong những tương tác của mô hình xoắn ốc có thể là yêu cầu hoàn thiện ở mỗi pha của dự án trong mô hình SDLC.



Hình 2-3: Mô hình phát triển phần mềm Xoắn ốc

Mô hình hướng đối tượng kết hợp chặt chẽ việc sử dụng lại các phần mềm cỡ lớn bằng cách kết hợp các mô đun có thể sử dụng lại được thành các hệ thống phần mềm mới. Trong trường hợp không có mô đun phần mềm nào có thể sử dụng được (theo thuật ngữ đối tượng hoặc thành phần) sẵn có, người phát triển có thể thực hiện một quy trình bản mẫu hoặc quy trình SDLC để hoàn thành việc phát triển một hệ thống mới.

2.2 Các hoạt động đảm bảo chất lượng phần mềm.

2.2.1 Đảm bảo chất lượng hợp đồng

Một hợp đồng tồi chắc chắn là khó có thể chấp nhận được. Từ quan điểm của SQA, một hợp đồng tồi – thường mô tả các yêu cầu không chặt chẽ và đưa ra kế hoạch cũng như ngân sách phi thực tế - thì sẽ dẫn đến một phần mềm có chất lượng tồi. Vì thế, một chương trình SQA cần được thực hiện để đảm bảo chất lượng phần mềm bằng cách rà soát lại những đề xuất ban đầu và sau đó là bản dự thảo hợp đồng (hoạt động “rà soát hợp đồng” bao gồm cả 2 hoạt động trên). Cả hai hoạt động rà soát trên là nhằm mục đích cải thiện ngân sách và thời gian biểu, là những cơ sở cho những đề nghị và hợp đồng sau này, đồng thời có thể biết được những rủi ro tiềm năng sớm (trong mục tiêu ban đầu và trong bản dự thảo hợp đồng).

Có khá nhiều tình huống có thể giúp một công ty phần mềm (“nhà cung cấp”) ký hợp đồng với một khách hàng. Phổ biến nhất là:

- Tham gia trong một cuộc đấu thầu

- Đưa ra bản phác thảo dựa trên yêu cầu đề xuất (RFP-Request For Proposal) của khách hàng
- Nhận một đặt hàng từ một khách hàng của công ty
- Nhận một yêu cầu từ bên trong hoặc từ phòng ban khác trong một tổ chức

Rà soát hợp đồng là một thành phần của SQA được nghĩ ra để hướng dẫn xem xét lại những bản dự thảo của những tài liệu đề xuất và hợp đồng. Nếu có thể, rà soát lại hợp đồng còn cung cấp sự giám sát những hợp đồng được thực hiện với những đối tác dự án tiềm năng và các nhà thầu phụ. Tiến trình rà soát có thể được chia thành hai giai đoạn:

- Giai đoạn 1: rà soát lại bản dự thảo đề xuất trước khi giao cho khách hàng tiềm năng (“rà soát bản dự thảo đề xuất”). Giai đoạn này rà soát lại bản dự thảo cuối cùng và những cơ sở đề xuất: những tài liệu yêu cầu của khách hàng, chi tiết yêu cầu thêm của khách hàng và dự diễn giải các yêu cầu, các ước lượng chi phí và tài nguyên, những hợp đồng hiện tại hoặc là những bản dự thảo hợp đồng của nhà cung cấp với các đối tác và nhà thầu phụ.
- Giai đoạn 2: rà soát lại bản dự thảo hợp đồng trước khi kí (“rà soát bản dự thảo hợp đồng”). Giai đoạn này rà soát lại bản dự thảo hợp đồng dựa trên đề xuất và sự hiểu biết (bao gồm cả những thay đổi) đã đạt được trong quá trình thương thảo hợp đồng.

Quá trình rà soát có thể bắt đầu khi những tài liệu dự thảo liên quan đã hoàn thành. Những cá nhân thực hiện rà soát phải xem xét kỹ lưỡng bản dự thảo trong khi đề cập đến một phạm vi toàn diện các đối tượng đang rà soát. Một danh sách kiểm tra là rất hữu ích cho việc đảm bảo xem xét hết các vấn đề liên quan.

Sau khi hoàn thành giai đoạn rà soát, việc cần thiết những sự thay đổi, cái thêm vào và sự hiệu chỉnh phải được thông báo bởi đội đề xuất (sau khi rà soát bản dự thảo đề xuất) và bởi ban phụ trách về luật pháp (sau khi rà soát lại bản dự thảo hợp đồng)

2.2.2 Đảm bảo chất lượng đặc tả

2.2.2.1 Chất lượng đặc tả

Đặc tả không có các hoạt động trước nó, và tất cả các hoạt động khác đều theo sau đặc tả. Vì vậy, nếu đặc tả không tốt thì phân tích, thiết kế cũng sẽ không tốt, và dẫn tới phát triển, bảo trì của 1 phần mềm kém hơn, hoặc thậm chí một phần mềm lỗi, và công sức bỏ ra để đảm bảo chất lượng thành lãng phí. Vì vậy, điều tối quan trọng là đặc tả phải đầy đủ và được định nghĩa tốt. Đặc tả thường gồm 6 nội dung sau:

- **Chức năng:** đặc tả các chức năng nào sẽ được phát triển